APPLIED DATA FINANCE
ADF Gen AI Hackathon 2026

AE Rule Engine

Configuration Management System

Startup & Operations Guide

VERSION	STACK	CLASSIFICATION
v2.0.0	**Spring Boot 3.2 + React 18**	**Internal & Confidential**

Contents

1 Prerequisites 3
2 Starting the Application 3
3 Verifying Everything Works 5
4 Using the Application 5
4.1 The Change Workflow — End to End 5
4.2 Environment Toggle (TEST / PROD) 6
4.3 Approval Status Reference 6
5 Key Pages at a Glance 7
6 Switching to PostgreSQL (Production) 8
7 API Quick Reference 8
8 Troubleshooting 9

1. Prerequisites

Before starting, confirm the following tools are installed on your machine. These are the only dependencies required — no additional services, databases, or cloud accounts are needed to run the application locally.

Tool	Minimum Version	Purpose	How to Verify
Java JDK	17 or higher	Runs the Spring Boot backend	java -version
Apache Maven	3.8 or higher	Builds the backend (wrapper included)	mvn -version
Node.js	18 or higher	Runs the React frontend	node --version
npm	9 or higher	Installs frontend packages	npm --version
Git	Any version	Optional — for source code access	git --version

Note — Maven Wrapper Included
The project ships with ./mvnw (Mac/Linux) and mvnw.cmd (Windows). You do not need Maven installed globally — the wrapper downloads the correct version automatically on first run.

2. Starting the Application

The application consists of two processes that must both be running: the Spring Boot backend (port 8080) and the React frontend (port 5173). Follow the three steps below.

1

Unzip and Prepare the Project

Unzip the downloaded package. You will see two folders inside: backend/ and frontend/

```
unzip ae-rule-engine-v7-final.zip
cd ae-rule-engine
```

2

Start the Backend — Spring Boot API (Port 8080)

Open Terminal 1. Navigate into the backend folder and start the Spring Boot server:

```
# Mac / Linux
cd backend
./mvnw spring-boot:run
```

```
# Windows
cd backend
mvnw.cmd spring-boot:run
```

First Run Download
On the very first run, Maven will download dependencies (~60 MB). This takes approximately 2 minutes. Subsequent starts take under 10 seconds.

Wait until you see this log line — the backend is ready:

```
Started RuleEngineApplication in 4.2 seconds
```

The backend automatically seeds all initial data on every startup:

- 20 AE decision rules (AE_PTSMI, AE_LTI, AE_TUSOFT_CV_SCORE, AE_RISK_RULE, and 16 more)
- 35 US state constraints — with CO, NC, NV, CA, PA, WA, SD initially set to OFF
- 7 affiliate channel configurations — CMPQ, CKPQ, QS, LT, ML, MO, CMACT
- 30 typed variables in the embedded engine variable registry

Resource	URL	Details
REST API Base URL	http://localhost:8080/api	All API endpoints are prefixed with /api
H2 Database Console	http://localhost:8080/h2-console	In-browser SQL query interface
H2 JDBC URL	jdbc:h2:mem:ruleenginedb	Use this in the H2 console login
H2 Credentials	Username: sa | Password: (leave blank)	Default in-memory credentials

3

Start the Frontend — React Application (Port 5173)

Open Terminal 2 (keep Terminal 1 running). Navigate into the frontend folder:

```
cd frontend
npm install
npm run dev
```

When ready, you will see:

```
VITE v5  ready in 600ms

  ➜  Local:   http://localhost:5173/
```

Open in Browser
Navigate to http://localhost:5173 in your browser. The AE Rule Engine dashboard loads immediately with live data from the backend.

2.1 Alternative — Start Both with One Command

On macOS or Linux, you can start both the backend and frontend simultaneously from the project root folder:

```
# From the ae-rule-engine/ root folder
(cd backend && ./mvnw spring-boot:run) &
(cd frontend && npm install && npm run dev)
```

Port Conflict Resolution
If port 8080 is already in use, stop the existing process with: kill $(lsof -ti:8080) on Mac/Linux. On Windows, open Task Manager → find the Java process → End Task. Alternatively, change server.port in backend/src/main/resources/application.properties.

3. Verifying Everything Works

Run through the following checks to confirm the backend and frontend are communicating correctly. All five checks should pass before proceeding.

#	What to Check	How	Expected Result
1	Dashboard loads	Open http://localhost:5173	Dashboard page renders with rule counts, activity feed, and stat cards
2	Rules are seeded	Click Rules in the sidebar	20 rules displayed — AE_PTSMI, AE_LTI, AE_DEDUPE_DAYS, AE_TUSOFT_CV_SCORE, etc.
3	State configs load	Click State Config in the sidebar	35 US states shown; CO, NC, NV, CA, WA, SD displaying OFF status
4	Backend API responds	Visit http://localhost:8080/api/dashboard/stats	JSON response with totalRules, pendingReviews, activeRules, and other counts
5	Analytics charts render	Click Analytics & Charts in sidebar	Bar charts, area charts, pie charts, and channel radar all visible with data

4. Using the Application

The AE Rule Engine manages the complete lifecycle of Affiliate Engine decision rules — from authoring through approval to production deployment. The sections below walk through the primary workflows.

4.1 The Change Workflow — End to End

Every configuration change follows the same governance path: Draft → Review → Approved → Production. This ensures no change reaches production without explicit Approver sign-off and a documented field-level diff.

1

Edit a Rule or Config Item

- Navigate to Rules, State Config, or Channel Config in the sidebar.
- Click the pencil icon (Edit) on any item in the table.
- Modify values in the form — for example, change a cutoff threshold or toggle a state on/off.
- Click Update. The item status changes to DRAFT and appears with a grey badge.

2

Submit for Approver Review

- The edited item now shows a purple Submit button.
- Click Submit — the item moves to PENDING REVIEW status.
- The Review Queue counter in the sidebar increments automatically.
- The Approver is now able to see and act on this change.

3

Approver Reviews with Field-Level Diff

- The Approver opens the Review Queue — all pending items are listed.
- Clicking the Diff button expands a field-level comparison: red (before) vs green (after).
- Every changed field is shown individually — not raw JSON — making it easy to spot exactly what changed.
- The Approver clicks Approve, or Reject with a mandatory written reason.

4

Promote to Production

- Go to Promote to Prod in the sidebar.
- Approved items appear in the Ready to Promote list.
- Click Compare to see a TEST vs PROD side-by-side comparison of all fields.
- Click Promote to PROD — the configuration is copied to the production environment and logged in the Version History.

4.2 Environment Toggle — TEST and PROD

Every list page (Rules, State Config, Channel Config) has a TEST / PROD toggle in the top-right corner. Switching the toggle shows configurations from that environment only. Rules and constraints exist independently per environment — promoting a rule creates a copy of the TEST record in the PROD environment. The two environments never share state automatically.

> **When to Use Each Environment**
> TEST is the working environment — create and modify rules here. PROD is the live environment — rules reach PROD only through the explicit Promote to PROD action after Approver sign-off. Never edit PROD records directly.

4.3 Approval Status Reference

Every rule, state constraint, and channel constraint has an approval status that controls visibility, editability, and promotion eligibility.

Status	Badge Colour	Meaning and Who Can Act
DRAFT	Grey	Saved but not yet submitted. Only the Lead Analyst who created it can view or edit the item. Not visible in the Review Queue.
PENDING REVIEW	Amber	Submitted and awaiting Approver action. The Lead Analyst can no longer edit. The Approver sees this in the Review Queue with the full field-level diff.
APPROVED	Green	Approved by the Approver. The item is now eligible for promotion to the PROD environment. No further edits are allowed until promoted.
REJECTED	Red	Rejected by the Approver with a mandatory written reason. The Lead Analyst must edit the item and resubmit. The rejection reason is preserved in the Version History Log.

5. Key Pages at a Glance

The AE Rule Engine has 16 pages accessible from the sidebar. The table below describes the purpose of each page and which role primarily uses it.

Sidebar Label	URL Path	What It Does	Primary Role
Dashboard	/	KPI cards, activity feed, quick navigation links	All

Sidebar Label	URL Path	What It Does	Primary Role
Analytics & Charts	/analytics	Recharts dashboards: rule health, approval rates, channel radar, governance score	All
Rules	/rules	View, search, and filter all 20+ AE decision rules by phase and status	Lead Analyst
State Config	/states	FEB state eligibility ON/OFF and loan constraint limits per state	Lead Analyst
Channel Config	/channels	Loan limits and APR ranges per affiliate channel	Lead Analyst
Review Queue	/reviews	All pending items with field-level red/green diff — approve or reject	Approver
Promote to Prod	/promote	TEST vs PROD side-by-side comparison and one-click promotion	Approver
Version History Log	/audit	Full audit trail: every create, update, approve, reject, promote with timestamps	All
UW Excel → Config	/uw-ingestion	Upload UW Analytics Excel spec — AI detects yellow cells and generates change suggestions	Lead Analyst
Impact Simulator	/simulator	Build an applicant profile and run it against active rules — see which rules fire	Lead Analyst
Rule Builder	/rule-builder	Guided 3-step popup builder: variable → operator → value, no code needed	Lead Analyst
Cutoff Tracker	/cutoff-tracker	Production cutoff table: one DB row per grade × channel × state combination	Lead Analyst
Cutoff Builder	/cutoff-builder	Create multi-dimensional cutoff groups and save them to the engine	Lead Analyst
Offer Config Loader	/offer-config	Upload offer logic Excel file to the embedded engine snapshot tables	Lead Analyst
Engine API Tester	/embedded	Raw API interface: save rules, execute rules, translate descriptions to conditions	Developer
Version Manager	/config-versions	Flyway-style semantic version history: full collision detection and rollback	All

6. Switching to PostgreSQL (Production)

By default, the application uses an H2 in-memory database. This means all data is reset every time the backend restarts — which is ideal for demos but not for persistent use. To retain data across restarts, switch to PostgreSQL.

6.1 Create a PostgreSQL Database

```
createdb ruleenginedb
# Or connect to psql and run:
CREATE DATABASE ruleenginedb;
```

6.2 Update application.properties

Edit the file at backend/src/main/resources/application.properties and replace the H2 block with the following:

```
# ── Database: PostgreSQL ──────────────────────────────────────
spring.datasource.url=jdbc:postgresql://localhost:5432/ruleenginedb
spring.datasource.driver-class-name=org.postgresql.Driver
spring.datasource.username=postgres
spring.datasource.password=your_password

# ── JPA / Hibernate ───────────────────────────────────────────
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

Schema Auto-Created
With ddl-auto=update, Hibernate automatically creates all required tables on first startup. No SQL migration scripts are needed. All 12 JPA entities will be created as tables in the ruleenginedb database.

7. API Quick Reference

All API endpoints are prefixed with http://localhost:8080. JSON is the content type for all request and response bodies. Authenticate by passing user or reviewedBy as a query parameter where required.

Config Management API

Method	Endpoint	Description
GET	/api/dashboard/stats	Returns dashboard counts: total rules, pending reviews, off states, active channels
GET	/api/dashboard/activity	Returns the 50 most recent audit log events

Method	Endpoint	Description
GET	/api/rules?env=TEST	Returns all rules for the specified environment (TEST or PROD)
GET	/api/rules/pending	Returns all rules in PENDING_REVIEW status
POST	/api/rules?user=lead-analyst	Creates a new rule in DRAFT status
PUT	/api/rules/{id}?user=lead-analyst	Updates an existing rule — resets status to DRAFT
POST	/api/rules/{id}/submit	Submits a DRAFT rule for Approver review
POST	/api/rules/{id}/review	Approves or rejects — body: { action, reviewer, comments }
POST	/api/rules/{id}/promote	Promotes an APPROVED rule to the PROD environment
GET	/api/constraints/states	Returns all state constraints for the specified environment
POST	/api/constraints/states	Creates or updates a state constraint
GET	/api/constraints/channels	Returns all channel constraints

Embedded Rule Engine API

All embedded engine endpoints are prefixed with /api/v1/embedded.

Method	Endpoint	Description
POST	/api/v1/embedded/rules/save	Saves a rule bundle — validates, compiles, snapshots, and updates the JVM cache
POST	/api/v1/embedded/rules/execute	Executes rules against a provided fact set — returns fired/passed/skipped results
POST	/api/v1/embedded/rules/validate	Dry-run validation — no DB writes, returns any validation errors
GET	/api/v1/embedded/rules/variables	Returns all 30 typed variables in the variable registry
GET	/api/v1/embedded/cutoffs/entries	Returns all active cutoff rows from the tracking table
GET	/api/v1/embedded/cutoffs/groups	Returns all distinct active cutoff group names
GET	/api/v1/embedded/offer-config/active	Returns the currently active offer config snapshot
GET	/api/v1/config-versions	Returns all config versions; filter by scope with ?scope=RULES
POST	/api/v1/config-versions/{id}/submit	Submits a DRAFT version for review
POST	/api/v1/config-versions/{id}/promote	Promotes an APPROVED version to the PROD environment

8. Troubleshooting

The following covers the most common issues encountered during startup and development. If a problem persists after applying the fix, restart both terminals and begin from Section 2.

Symptom	Cause	Fix
"Port 8080 already in use"	Another process is using port 8080	Mac/Linux: kill $(lsof -ti:8080) Windows: Task Manager → find Java → End Task Or change server.port in application.properties
Frontend shows blank screen or loading spinner indefinitely	Backend is not running or CORS error	Ensure the backend started successfully in Terminal 1. Open browser DevTools → Console and check for errors. Confirm http://localhost:8080/api/dashboard/stats returns JSON.
"mvnw: Permission denied"	Maven wrapper not executable on Mac/Linux	Run: chmod +x mvnw Then re-run: ./mvnw spring-boot:run
"npm install" fails with dependency errors	Node version mismatch or stale cache	Run: npm install --legacy-peer-deps Or delete the node_modules/ folder and retry: rm -rf node_modules && npm install
Rules, states, or channels show as empty	Data seed did not run or completed with errors	Check the backend terminal for "AE Rule Engine data seeded successfully". If not present, look for a stack trace above it and restart the backend.
H2 Console login fails	Incorrect JDBC URL entered	JDBC URL must be exactly: jdbc:h2:mem:ruleenginedb Username: sa Password: (leave blank). Do not modify the URL.
Analytics charts show no data	No rules in the database	Confirm the backend is seeded (check Rules page). Charts are driven by the /api/rules and /api/dashboard/stats endpoints — both must return data.
AE Copilot chatbot not visible	Frontend did not load the component	Hard-refresh the browser (Ctrl+Shift+R or Cmd+Shift+R). The chatbot icon appears in the bottom-right corner of every page.
Impact Simulator returns no rules	Embedded engine has no active rule bundle	The Simulator uses a built-in fallback simulation if no rules are loaded. To use live rules, open Rule Builder → add rules → Save. The Simulator will then use those rules.

AE Rule Engine — v2.0.0

Configuration Management System · Applied Data Finance · ADF Gen AI Hackathon 2026

Internal & Confidential · Not for external distribution